

COM3105 Advanced Algorithms

Homework 3

Joffray Hargreaves

Question 1

(4 points)

Consider a stream $a_1, \dots, a_m$ where each $a_i \in \{1, \dots, n\}$.

Give a deterministic streaming algorithm that maintains the sum $a_1 + \dots + a_m$ using $\mathcal{O}(\log m + \log n)$ space.

Answer

Algorithm:

Let S_i be the sum of the the stream at step i , for $i = 0, \dots, m$.

- Initialise $S_0 = 0$.
- For each incoming element a_i , get the sum at time step i by $S_i = S_{i-1} + a_i$
- At any point, the current sum of the stream is given by S_i .
- After processing all m elements, $S_i = S_m$, i.e. the sum of all elements $a_1, \dots, a_m$.

This algorithm correctly maintains the sum of the stream as each incoming element is added to the current sum.

Space Complexity Analysis:

In the worst case, the value of any element a_i is n .

So we consider the following worst case run of the algorithm, where each $a_i = n$:

- Initially, $S_0 = 0$,
- $S_1 = S_0 + n = n$,
- $S_2 = S_1 + n = n + n = 2 \times n$,
- ...
- $S_i = S_{i-1} + n = i \times n$,
- ...
- $S_m = S_{m-1} + n = m \times n$

Therefore, the maximum value that S_i can take is $m \times n$.

To store $m \times n$, we need $\log(m \times n) = \log m + \log n$ bits.

Hence, the space complexity of the algorithm is $\mathcal{O}(\log m + \log n)$.

Question 2

(6 points)

Consider a stream $a_1, \dots, a_m$ where each $a_i \in \{1, \dots, n\}$.

Give a randomised streaming algorithm which approximates the sum $\sum a_1 + \dots + a_m$ using $\mathcal{O}(\log \log m + \log n)$ space.

Give a rigorous analysis of the algorithm in terms of its quality and space complexity.

Answer

Algorithm:

I use an approach which uses the same concept as Morris's algorithm.

We keep a counter which has value X_i at time step i .

- **Initialisation:** Start with the counter $X_0 = 0$.
- **Processing:** For each integer a_i in the stream:
 - We treat the value a_i as a sequence of a_i separate increments of 1 (i.e. if $a_i = 4$ we consider it as 1, 1, 1, 1).
 - Repeat the following step a_i times:
 - ◊ with probability $\frac{1}{2^X}$, increment the counter X by 1 (i.e., $X \leftarrow X + 1$).
 - ◊ with probability $1 - \frac{1}{2^X}$, do nothing.
- **Output:** When asked for the sum (i.e. after processing m elements), output $\hat{S} = 2^X - 1$, where $\hat{S}$ is an estimate of the true sum S .

Expectation Analysis:

Let S_i be the true sum of all elements up to step i , and X_i be the value of the counter at step i . We prove by induction that $\mathbb{E}[2^{X_i}] = S_i + 1$.

- **Base Case:** Initially $S_0 = 0$ and $X_0 = 0$. $\mathbb{E}[2^0] = 1$. Since $S_0 + 1 = 0 + 1 = 1$, the base case holds.
- **Inductive Case:**
 - Assume $\mathbb{E}[2^{X_{i-1}}] = S_{i-1} + 1$ holds before an update.
 - Consider a single increment from a_i (adding 1 to the sum).
 - Let X_i be the value of the counter after this update.
 - ◊ With probability $1/2^{X_{i-1}}$, X_i becomes $X_{i-1} + 1$. Hence, 2^{X_i} becomes $2^{X_{i-1}+1}$.
 - ◊ With probability $1 - 1/2^{X_{i-1}}$, X_i stays X_{i-1} . Hence, 2^{X_i} stays $2^{X_{i-1}}$.
 - Expectation of 2^{X_i} given X_{i-1} :
 - ◊ $\mathbb{E}[2^{X_i}] = \sum_{j=0}^{\infty} \mathbb{P}[X_{i-1} = j] \cdot \mathbb{E}[2^{X_i} \mid X_{i-1} = j]$
 - We can simplify the right side:
 - ◊ $\mathbb{E}[2^{X_i} \mid X_{i-1} = j]$
 - ◊ $= \mathbb{P}(\text{we increase the counter}) \times 2^{j+1} + \mathbb{P}(\text{we do not increase the counter}) \times 2^j$
 - ◊ $= \frac{1}{2^j} \cdot 2^{j+1} + \left(1 - \frac{1}{2^j}\right) \cdot 2^j$
 - ◊ $= 2 + 2^j - 1$

$$\diamond = 2^j + 1$$

◦ We then substitute this into the expectation:

$$\diamond \mathbb{E}[2^{X_i}] = \sum_{j=0}^{\infty} \mathbb{P}[X_{i-1} = j] \cdot (2^j + 1)$$

$$\diamond = \sum_{j=0}^{\infty} \mathbb{P}[X_{i-1} = j] \cdot 2^j + \sum_{j=0}^{\infty} \mathbb{P}[X_{i-1} = j]$$

$$\diamond = \mathbb{E}[2^{X_{i-1}}] + 1$$

◦ By the inductive hypothesis above, $\mathbb{E}[2^{X_{i-1}}] = S_{i-1} + 1$.

◦ Therefore, $\mathbb{E}[2^{X_i}] = (S_{i-1} + 1) + 1 = S_i + 1$.

◦ So the claim holds for S_i after one increment.

Therefore, after processing all m elements, we have $\mathbb{E}[2^{X_m}] = S_m + 1$.

Quality Analysis:

First we get the variance of 2^{X_m} .

Note: I used <https://madscience.ucsd.edu/2020/notes/lec4and5.pdf> for the $\mathbb{E}[2^{2 \cdot X_m}]$ simplification step.

$$\begin{aligned} \text{Var}[2^{X_m}] &= \mathbb{E}[2^{2 \cdot X_m}] - (\mathbb{E}[2^{X_m}])^2 \\ &= \frac{3}{2}S_m^2 + \frac{3}{2}S_m + 1 - (S_m + 1)^2 \\ &= \frac{3}{2}S_m^2 + \frac{3}{2}S_m + 1 - (S_m^2 + 2S_m + 1) \\ &= \frac{S_m^2}{2} - \frac{S_m}{2} \leq \frac{S_m^2}{2} \end{aligned}$$

We note our estimate of the sum as $\hat{S} = 2^{X_m} - 1$.

Chebyshev's inequality is defined as:

$$\mathbb{P}(|X - \mu| > \lambda) \leq \frac{\text{Var}(X)}{\lambda^2}$$

Applying Chebyshev's inequality to our estimate $\hat{S}$:

$$\mathbb{P}(|\hat{S} - S_m| \geq \epsilon S_m) \leq \frac{\text{Var}[\hat{S}]}{(\epsilon S_m)^2} = \frac{\text{Var}[2^{X_m}]}{(\epsilon S_m)^2} \leq \frac{\frac{S_m^2}{2}}{\epsilon^2 S_m^2} = \frac{1}{2\epsilon^2}$$

Therefore, the probability that our estimate $\hat{S}$ deviates from the true sum S_m by at least a factor of ϵS_m is at most $\frac{1}{2\epsilon^2}$.

This only holds for $\epsilon > \frac{1}{\sqrt{2}}$.

To increase this bound, we can run k independent copies of the algorithm in parallel and take the median of their sums.

Space Complexity Analysis:

In the worst case, all elements $a_i = n$.

So the maximum possible sum is $S_{\max} = m \cdot n$.

The counter X is kept, where $\hat{S} = 2^X - 1$ on expectation, so the largest X is

$$X_{\max} = \log_2(S_{\max}) = \log_2(mn).$$

To store X in this worst case, the number of bits required is:

$$\log_2(X_{\max}) = \log_2(\log_2(mn)).$$

Therefore, the expected space required for the counter X is:

$$\mathcal{O}(\log \log m + \log \log n).$$

As well as this we need to space to process a_i , as we iterate through it in increments of 1. In the worst case, each $a_i = n$. To store a_i as we iterate through it in this worst case, we need $\mathcal{O}(\log n)$ bits.

Combining these two space complexities:

$$\mathcal{O}(\log \log m + \log \log n) + \mathcal{O}(\log n) = \mathcal{O}(\log \log m + \log \log n + \log n).$$

As $\log n$ is exponentially larger than $\log \log n$, we simplify to:

$$\mathcal{O}(\log \log m + \log n).$$

Hence, the space complexity of the algorithm is $\mathcal{O}(\log \log m + \log n)$.
